SYSTEM DESIGN DOCUMENT

TaskAnalyzer — documento de arquitetura e desenho do sistema

Projeto: TaskAnalyzer
Aluno: Guilherme Eduardo Rodrigues Krinski
Curso: Análise e Desenvolvimento de Sistemas
Versão: 1.0 — documento elaborado a partir do código e dos testes apresentados

1. Objetivo

O TaskAnalyzer é um módulo Python que recebe uma lista de tarefas em memória e calcula indicadores de conclusão e atraso. A implementação atual não possui interface gráfica, API, persistência em arquivos ou banco de dados. A execução é feita por chamada à função `analyze_tasks`, e a validação dos dados utiliza a exceção específica `TaskValidationError`.

2. Escopo funcional implementado

A função retorna as seguintes métricas:

Campo de saída	Significado
total_tarefas	Quantidade de itens recebidos na lista.
total_concluidas	Quantidade de tarefas com status `concluida`.
total_pendentes	Quantidade de tarefas com status `pendente`.
tempo_medio_conclusao_horas	Média do tempo entre criação e conclusão das tarefas concluídas.
tempo_medio_conclusao_horas_por_prioridade	Média do tempo de conclusão agrupada pelo texto da prioridade.
taxa_atraso_percentual	Percentual de tarefas concluídas após o prazo, sobre o total de tarefas concluídas.

A taxa de atraso usa o campo adicional `data_prazo`, exigido para tarefas concluídas. Essa regra está registrada na especificação do projeto como uma decisão proposta, pois o contrato inicial de tarefa não apresentava um campo de prazo; sua validação pelo professor permanece pendente.

3. Arquitetura e organização

A solução segue uma estrutura modular simples, adequada ao escopo atual:

Componente	Responsabilidade
`src/task_analyzer.py`	Contém a exceção de domínio e a função principal de análise, incluindo validações e cálculos.
`tests/test_harness.py`	Contém testes automatizados de aceitação com pytest para métricas, dados inválidos e casos extremos.
`specs/task_analyzer_spec.md`	Registra objetivo, estrutura dos dados, métricas, validações e regra proposta para atraso.

Componente	Responsabilidade
`CONTEXT_RULES.md`	Define restrições e orientações de implementação e governança de contexto.
`requirements.txt`	Declara pytest como dependência Python para execução dos testes.
`Dockerfile` / `docker-compose.yml`	Permitem construir uma imagem com Python 3.11 e executar a suíte de testes em contêiner.
`.github/workflows/ci.yml`	Configura execução automatizada dos testes em eventos de push e pull request definidos no workflow.

4. Fluxo de processamento

1. A função recebe a lista de tarefas.
2. Confere se a entrada é uma lista e se cada item é um dicionário.
3. Verifica presença dos campos obrigatórios, título, prioridade e status.
4. Converte as datas ISO 8601 para objetos datetime e verifica consistência temporal e de fuso horário.
5. Para tarefas pendentes, exige `data_conclusao = None` e contabiliza a pendência.
6. Para tarefas concluídas, exige data de conclusão e prazo, calcula a duração e identifica atrasos.
7. Calcula as métricas e devolve um dicionário. Se não houver tarefas concluídas, as médias e a taxa de atraso correspondentes retornam 0.0.

5. Dados e regras de validação

Cada tarefa usa os campos `id`, `titulo`, `prioridade`, `data_criacao`, `data_conclusao` e `status`. Para tarefas concluídas, a implementação também exige `data_prazo`.

As datas devem ser strings aceitas por `datetime.fromisoformat`. A data de conclusão não pode ser anterior à criação; o prazo também não pode anteceder a criação. As datas devem utilizar o mesmo padrão de fuso horário conforme a verificação implementada. Tarefas pendentes não podem apresentar data de conclusão, e tarefas concluídas precisam apresentar conclusão e prazo.

Os status aceitos são exatamente `concluida` e `pendente`. Títulos e prioridades devem ser textos não vazios. As falhas previstas são comunicadas por `TaskValidationError`, uma subclasse de `ValueError`.

6. Estratégia de testes e evidências

O arquivo de testes apresentado cobre total de tarefas, contagem de concluídas e pendentes, média geral e por prioridade, ordem inválida das datas, lista vazia, ausência de tarefas concluídas, taxa de atraso em cenários de 50% e 0%, campos ausentes, formato de data inválido, prazo anterior à criação, item que não é dicionário, status inválido, entrada que não é lista, título/prioridade vazios, inconsistência entre status e data de conclusão, tarefa concluída sem prazo e fusos horários diferentes.

No histórico informado pelo desenvolvedor, a suíte foi executada em contêiner com Python 3.11 e 22 testes passaram; a cobertura de código reportada para `src/task_analyzer.py` foi de 100%. Esses resultados são registros fornecidos durante o desenvolvimento e devem ser atualizados caso novas alterações sejam feitas.

7. Execução e integração contínua

A imagem Docker parte de `python:3.11-slim`, instala as dependências declaradas em `requirements.txt` e executa pytest. O Compose define o serviço `task-analyzer-tests`. O workflow GitHub Actions configura Python 3.11, instala dependências e executa `python -m pytest -v` em eventos de push e pull request configurados.

8. Limitações e decisões pendentes

• A aplicação é um módulo de análise em memória; não há persistência, interface web, API ou armazenamento.
• A regra de atraso depende de `data_prazo`, campo adicional cuja homologação deve ser confirmada.
• A implementação valida a presença de `id`, mas não valida seu tipo.
• Os testes apresentados demonstram as situações listadas, mas não substituem a revisão humana do contrato de requisitos.
• O relatório descreve o código e os arquivos informados; recomenda-se conferir o conteúdo final do repositório antes da entrega.

9. Governança e revisão humana

A especificação e as regras de contexto devem funcionar como limites para mudanças geradas com apoio de IA. Alterações em métricas, campos, dependências ou comportamento precisam ser comparadas ao contrato e aos testes. A revisão humana deve confirmar especialmente a definição da taxa de atraso e aprovar qualquer

mudança de regra de negócio antes de incorporá-la.

Documento preparado com base na especificação, no código-fonte e no arquivo de testes compartilhados pelo aluno. Não representa homologação formal do professor.